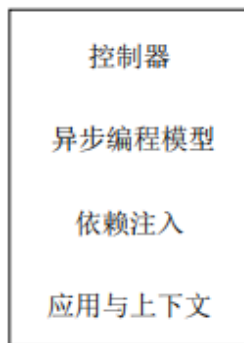


Web后端开发技术



后端服务架构



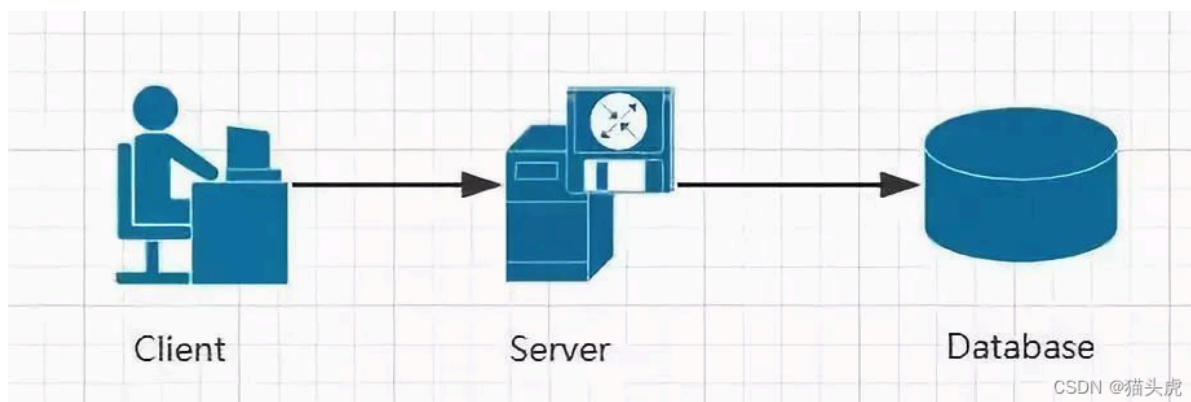
Node.js Midway框架



实验: 构建一个Web App 后端

1.Web应用架构

1.1 单体架构



优点: 清晰直接

- 一站式解决方案
- 迅速部署
- 减少分布式复杂性

缺点: 规模的挑战

- 难以应对复杂性
- 扩展的局限性
- 技术迭代困难

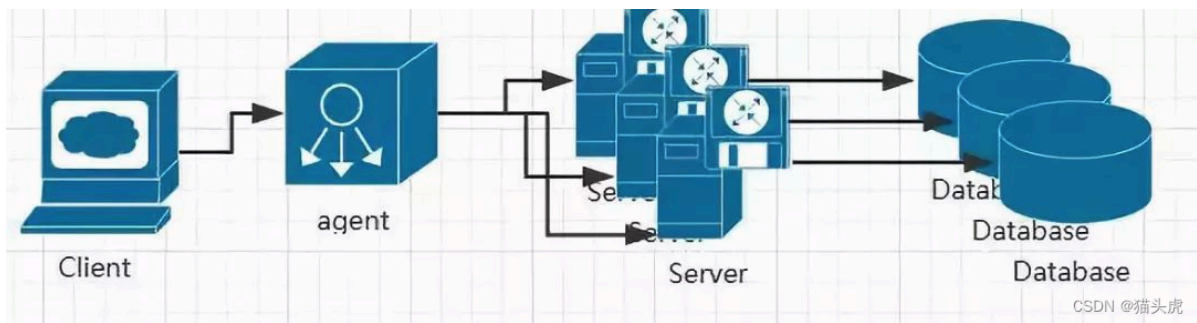
使用场景: 小而美

- 创业初期
- 内部管理系统

技术选型: 稳定/高效

- Java/Spring MVC
- Python/Django

1.2 分布式架构



优点：弹性和自由

- 灵活的扩展性
- 模块独立
- 资源优化

缺点：挑战与思考

- 网络通信的挑战
- 数据一致性
- 运维复杂度

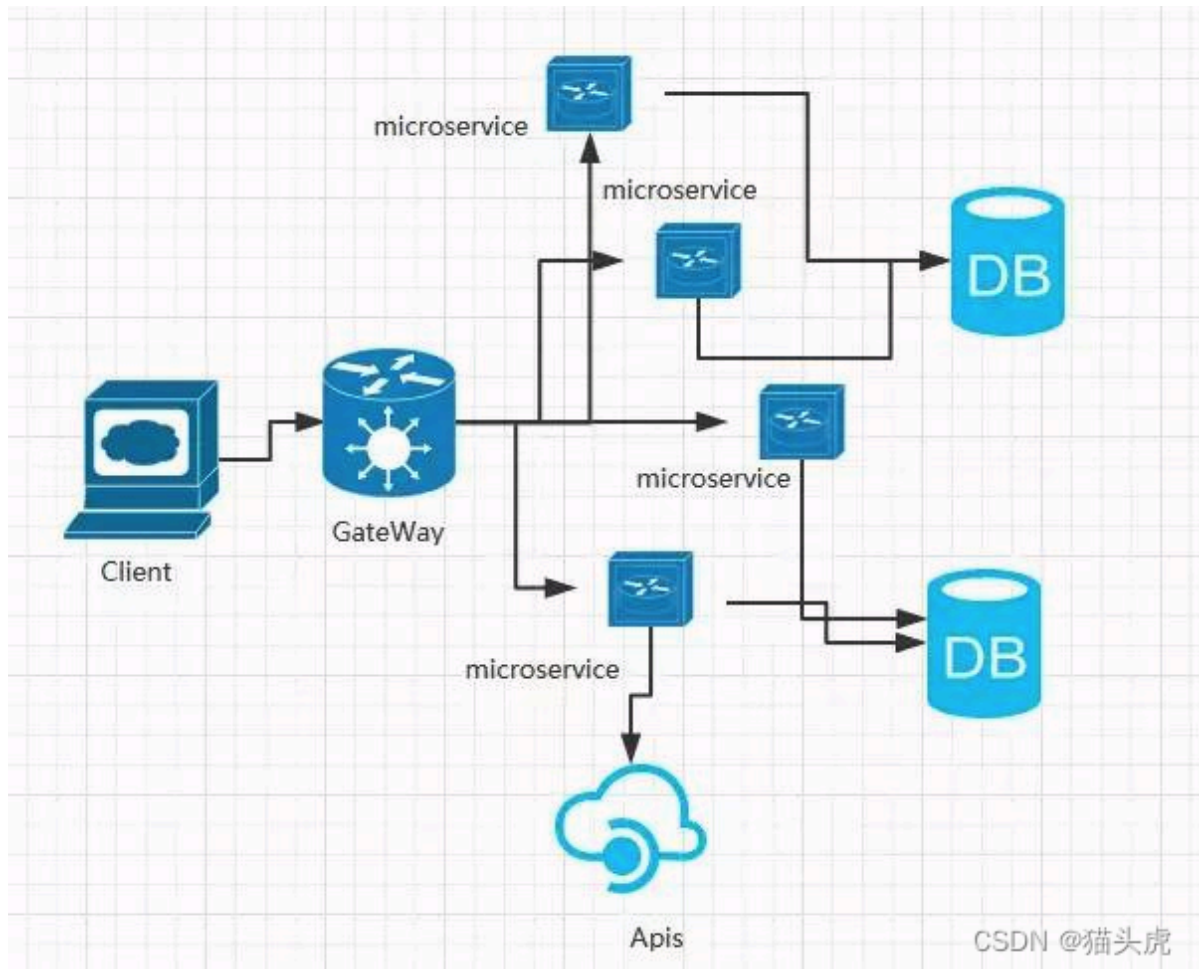
使用场景：广阔的应用天地

- 复杂业务逻辑
- 高并发处理
- 微服务的基石

技术选型：多元/兼容

- Java/Java EE
- Go
- Python+Flask/Django

1.3 微服务架构



优点：细胞分裂般的灵活与生命力

- 灵活而独立的扩展
- 多样的技术栈选择
- 敏捷的迭代与部署

缺点：复杂性

- 运维的挑战
- 网络依赖
- 数据一致性

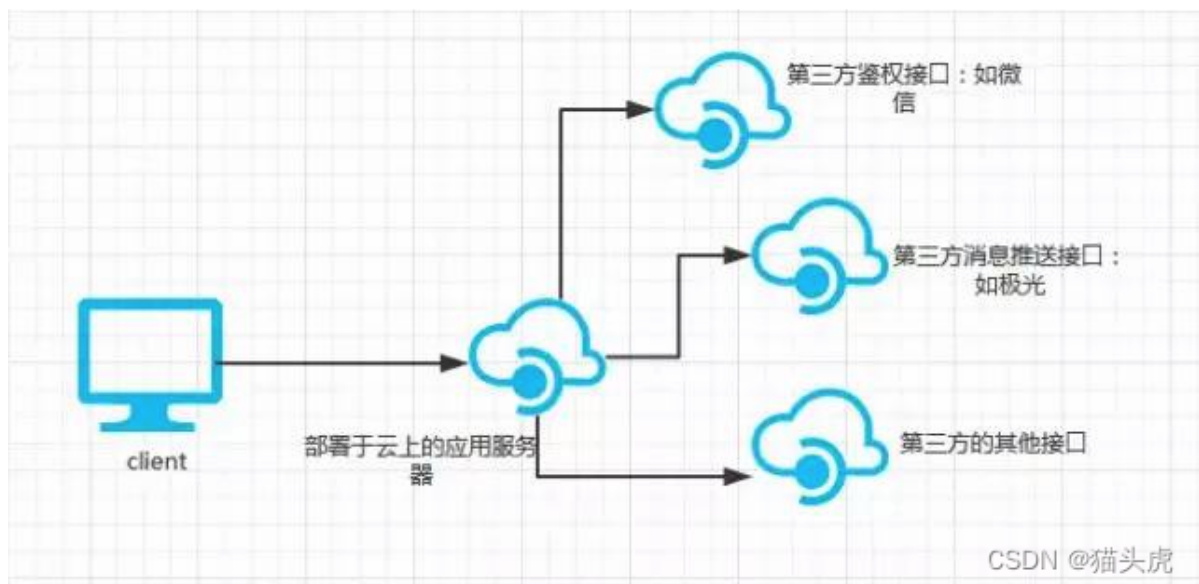
使用场景：细粒度的架构世界

- 复杂业务解构
- 高效的开发迭代

技术选型：无限的可能

- Node.js
- Spring Boot+Spring Cloud
- Docker+Kubernetes

1.4 Serverless架构



优点：轻松游走在云端的便利

- 极大降低运维负担
- 弹性扩展和计费
- 快速部署和迭代

缺点：自由与束缚并存

- 高度依赖云平台
- 冷启动延迟
- 网络延迟

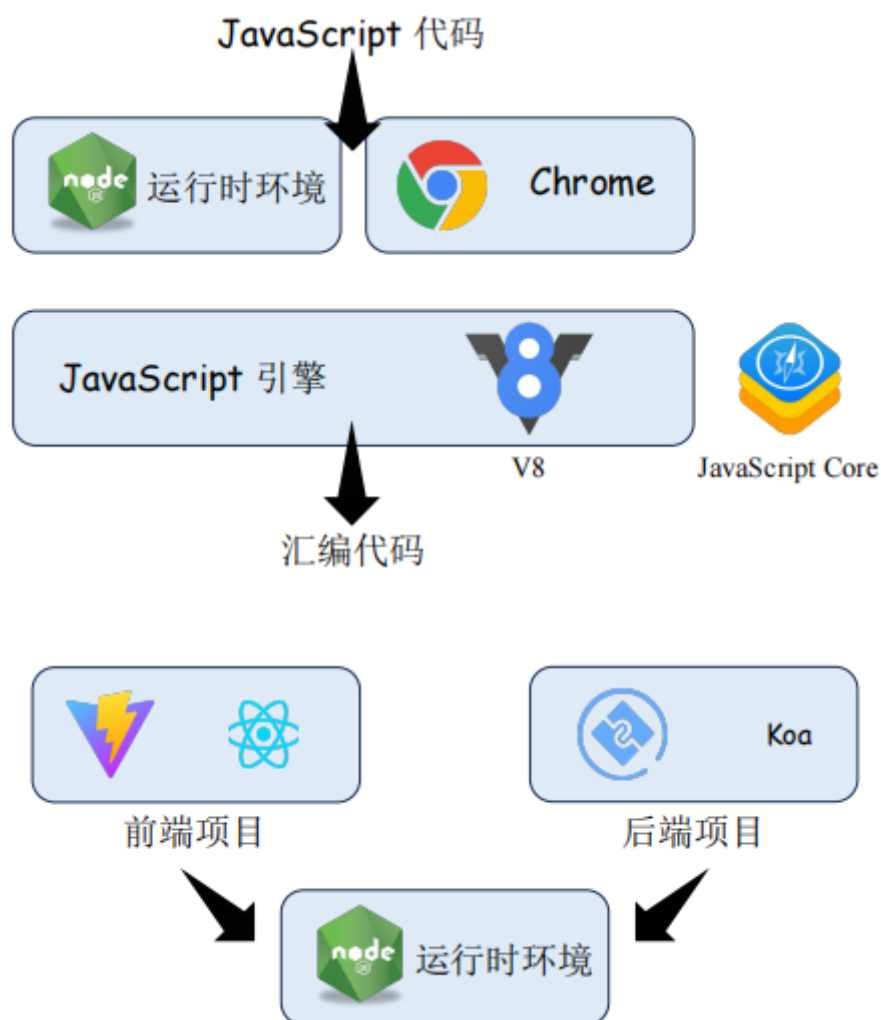
使用场景：适应性选择

- 事件驱动的计算
- 轻量级的Web应用
- 临时性的大规模计算

技术选型：多元的技术搭配

- AWS Lambda
- Azure Functions
- Google Cloud Functions

2.Node.js



上述两张图描述了Node.js的一些基本作用。

JS引擎：V8是支持Google Chrome的JS引擎，即它解析和执行JS代码。DOM和其他Web平台API由浏览器提供。

JS引擎独立于托管它的浏览器。这一特性促成了Node.js的崛起。

其他JS引擎：

- Firefox具有SpiderMonkey
- Safari有JavaScriptCore(Nitro)
- Edge最初基于Chakra，但最近改为使用Chromium重建和V8引擎

3.TypeScript

TS是JS的一个超集，支持ECMAScript 6标准，由微软开发的自由和开源的编程语言，设计目标是开发大型应用，它可以编译成纯JS，编译出来的JS可以运行在任何浏览器上。

TS是一种给JS添加特性的语言扩展，增加的功能包括：

- 类型批注和编译时类型检查
- 类型推断
- 类型擦除
- 接口

- 枚举
- Mixin
- 泛型编程
- 名字空间
- 元组
- Await

[TS教程](#)

[TS Vscode 开发环境详解](#)

TS学习重点:

- 数据类型

```
1 number,boolean,string
2 number[],Array<number>
3 any
4 interface,union
```

- 函数

```
1 function f1(param_1:number,param_2:string):string{
2     return 'hello';
3 }
```

- class

```
1 export class Task{
2
3     name:string
4     description:string
5     createdAt:Date
6     constructor(name:string,description:string,createdAt:Date){
7         this.name = name
8         this.description = description
9         this.createdAt = createdAt
10    }
11    public getName():string{
12        return this.name;
13    }
14 }
```

- async函数: 异步函数返回的是一个未来某一时刻执行完成的结果

```

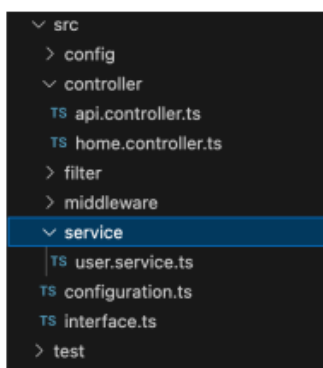
1 @Controller('/')
2 export class HomeController {
3     @Get('/')
4     async home(): Promise<object> {
5         return {
6             message: 'hello web development!',
7         };
8     }
9 }

```

[理解异步函数async和await的用法_async await用法-CSDN博客](#)

[TS Promise详解](#)

4.后端代码结构



- Controller:负责接收和响应外部请求

IoC&DI:

- 控制反转即IoC(Inversion of Control),它把传统上由程序代码直接操控的对象的调用权交给容器,通过容器来实现对象组件的装配和管理。
- 控制反转主要实现方式有两种: 依赖注入和依赖查找。
- 依赖注入 (Dependency Injection):组件之间的依赖关系由容器在系统运行期来决定,也就是由容器动态地将某种依赖关系的目标对象实例注入到应用系统中的各个关联的组件之中。组件不做定位查询,只提供普通的Java方法让容器去决定依赖关系。依赖注入分为接口注入、Setter方法注入、构造器注入。

[控制反转\(IoC\)与依赖注入\(DI\)详解_控制反转和依赖注入的概念-CSDN博客](#)

文件上传:

```
$ npm i @midwayjs/upload@3 --save
```

```

import { Configuration } from '@midwayjs/core';
import * as upload from '@midwayjs/upload';

@Configuration({
  imports: [
    // ...other components
    upload
  ],
  // ...
})
export class MainConfiguration {}

```

configuration.ts

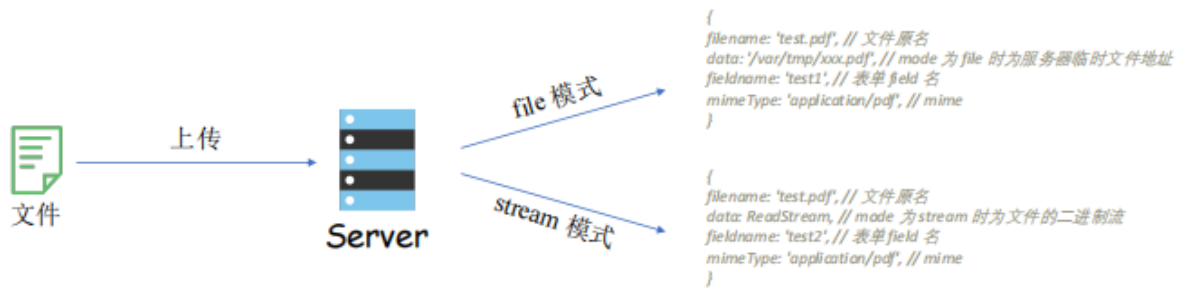
```

export default {
  // ...
  upload: {
    // mode: UploadMode, 默认为file, 即上传到服务器临时目录, 可以配置为 stream
    mode: 'file', // 'stream'
    // fileSize: string, 最大上传文件大小, 默认为 10mb
    fileSize: '10mb',
    // whitelist: string[], 文件扩展名白名单
    whitelist: uploadWhitelist.filter(ext => ext !== '.pdf'),
    // tmpdir: string, 上传的文件临时存储路径
    tmpdir: join(tmpdir(), 'midway-upload-files'),
    // cleanTimeout: number, 上传的文件在临时目录中多久之后自动删除, 默认为 5 分钟
    cleanTimeout: 5 * 60 * 1000,
    // base64: boolean, 设置原始body是否是base64格式, 默认为false, 一般用于腾讯云的兼容
    base64: false,
    // 仅在匹配路径到 /api/upload 的时候去解析 body 中的文件信息
    match: /\api\/upload/,
  },
}

```

config.xx.ts

file&stream:



Callback Function:

```
1 | setCount((count) => count + 1)
```

setCount以回调函数作为参数。

Koa: Koa是一个新的Web框架，由Express原班人马打造，致力于成为Web应用和API开发领域的一个更小、更富有表现力、更健壮的基石。通过利用async函数，Koa帮你丢弃回调函数，并有力地增强错误处理。

[Koa中文文档](#)

[Koa框架教程](#)

[koa-router的使用](#)

Postman: 是一个用于API开发和测试的工具，可以帮助您发出请求、编写测试脚本、构建和管理API、发布文档等。

[PostMan中文文档](#)